جامعة عفت

EFFAT UNIVERSITY

CS 3081 - Artificial Intelligence

Assignment#3 - Fall 2024

Maram Alhusami and Amani Albarazi

Instructor: **Akila Sarirete**

Date Last Edited: November 23, 2024

Contents

1 Introduction

In this assignment, we solved the map coloring problem of Saudi Arabia as the problem domain using Constraint Satisfaction Problems (CSP). And the 8-Queens problem using a Genetic Algorithm. We will be addressing each problem and explaining our solution using the required search algorithm, then concluding their effectiveness and what we learned from this assignment.

2 Map Coloring Problem of Saudi Arabia Using Constraint Satisfaction Problem (CSP)

2.1 Overview

The goal is to color a map of Saudi Arabia such that no two adjacent regions have the same color. Each region must be colored using one of three colors: Red, Green, or Blue.

2.2 Map Coloring Problem Formulations

- Variables: KSA Regions.
- Domains: Colors Red, Green, Blue.
- Constraints: No two adjacent regions should have the same color.

2.3 Code of the Map Coloring Problem: CSP Implementation

- **Code to Find the Path**

```python
from csp import CSP, backtracking_search, AC3

# Define KSA regions and their adjacency
regions = ['Najran', 'Jazan', 'Asir', 'Makkah', 'Riyadh', 'Eastern', 'Qassim', 'Hail', 'Tabuk', 'Madinah', 'Northern', 'Al-Baha']
adjacencies = {
    'Najran': ['Jazan', 'Asir'],
    'Jazan': ['Najran', 'Asir', 'Makkah'],
    'Asir': ['Najran', 'Jazan', 'Makkah', 'Al-Baha'],
    'Makkah': ['Jazan', 'Asir', 'Riyadh', 'Eastern', 'Al-Baha'],
    'Riyadh': ['Makkah', 'Eastern', 'Qassim', 'Hail'],
    'Eastern': ['Makkah', 'Riyadh'],
    'Qassim': ['Riyadh', 'Hail'],
    'Hail': ['Riyadh', 'Qassim', 'Tabuk'],
    'Tabuk': ['Hail', 'Madinah'],
    'Madinah': ['Tabuk', 'Makkah'],
    'Northern': ['Makkah'],
    'Al-Baha': ['Asir', 'Makkah']
}

# Define the domains (colors)
colors = ['Red', 'Green', 'Blue']

# Define the Map Coloring CSP problem
def map_coloring_csp():
    # Variables (regions)
    variables = regions

    # Domains (colors) - each region gets the list of colors
    domains = {region: colors for region in regions}

    # Neighbors (adjacencies) - adjacency relationships
    neighbors = adjacencies

    # Constraints: No two adjacent regions should have the same color
```

```
    def adjacent_constraint(Xi, x, Xj, y):
        return x != y  # regions should not have the same color

    # Create a CSP instance with the required parameters
    csp = CSP(variables, domains, neighbors, adjacent_constraint)

    return csp

# Create the CSP problem instance
csp_problem = map_coloring_csp()

# Apply Arc Consistency (AC-3)
AC3(csp_problem)

# Solve the problem using Backtracking Search
solution = backtracking_search(csp_problem)

# Print the solution
if solution:
    for region, color in solution.items():
        print(f"{region}: {color}")
else:
    print("No solution found.")
```

Listing 1: Map Coloring CSP in Python

- **Code of The Map Coloring**

```
import matplotlib.pyplot as plt

# Define the regions and their assigned colors from the CSP solution
solution = {
    'Najran': 'Red',
    'Jazan': 'Green',
    'Asir': 'Blue',
    'Makkah': 'Red',
    'Riyadh': 'Green',
    'Eastern': 'Blue',
    'Qassim': 'Red',
    'Hail': 'Blue',
    'Tabuk': 'Red',
    'Madinah': 'Green',
    'Northern': 'Green',
    'Al-Baha': 'Green'
}

# Coordinates for plotting the regions (these are simplified coordinates, not
    exact geographic positions)
region_coordinates = {
    'Najran': (18.0, 44.0),
    'Jazan': (17.0, 42.0),
    'Asir': (18.5, 42.5),
    'Makkah': (21.4, 39.8),
    'Riyadh': (24.7, 46.7),
    'Eastern': (26.0, 49.0),
    'Qassim': (26.0, 43.5),
    'Hail': (27.5, 41.5),
    'Tabuk': (28.0, 36.5),
```

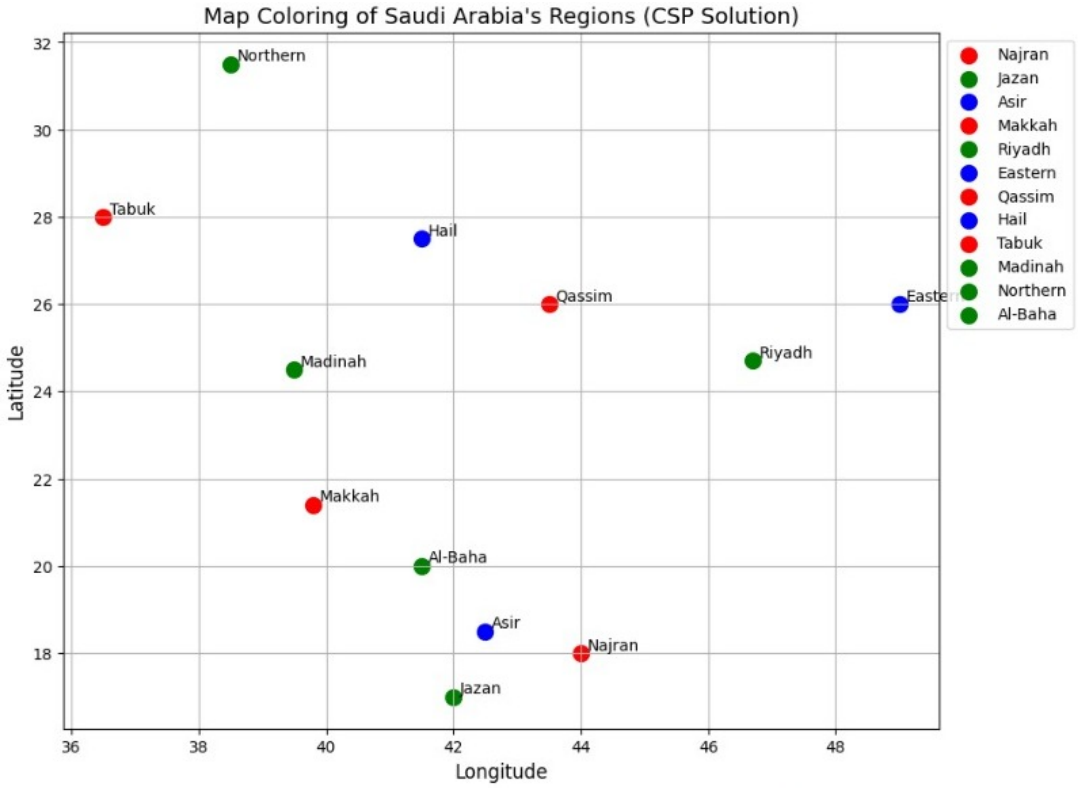
```
    'Madinah': (24.5, 39.5),
    'Northern': (31.5, 38.5),
    'Al-Baha': (20.0, 41.5)
}

# Plotting the map with region labels and assigned colors
plt.figure(figsize=(10, 8))

# Plot each region with its corresponding color
for region, (lat, lon) in region_coordinates.items():
    plt.scatter(lon, lat, color=solution[region], s=100, label=region)  # Points for regions
    plt.text(lon + 0.1, lat, region, fontsize=10, ha='left', va='bottom')  # Region label

# Adding title and axis labels
plt.title("Map Coloring of Saudi Arabia's Regions (CSP Solution)", fontsize=14)
plt.xlabel("Longitude", fontsize=12)
plt.ylabel("Latitude", fontsize=12)
plt.legend(loc='upper left', bbox_to_anchor=(1, 1))

# Show plot
plt.grid(True)
plt.show()
```

Listing 2: Map Visualization Using Matplotlib

2.4 Path Found

```
Najran: Red
Jazan: Green
Asir: Blue
Makkah: Red
Riyadh: Green
Eastern: Blue
Qassim: Red
Hail: Blue
Tabuk: Red
Madinah: Green
Northern: Green
Al-Baha: Green
```

2.5 Visualization of The Path Found



2.6 Explanation of the Solution Approach

2.6.1 Overview

The Map Coloring Problem is solved using a **Constraint Satisfaction Problem (CSP)** approach. The goal is to assign colors to regions of Saudi Arabia such that no two adjacent regions share the same color. Below is the breakdown of the approach:

2.6.2 CSP Formulation

- **Variables:** Each region in Saudi Arabia is treated as a variable in the CSP. These regions include: Najran, Jazan, Asir, Makkah, Riyadh, Eastern, Qassim, Hail, Tabuk, Madinah, Northern, and Al-Baha.
- **Domains:** The domain for each variable (region) consists of three possible colors: `Red`, `Green`, and `Blue`. Each region can be assigned any of these three colors.
- **Neighbors:** The adjacency relations between regions define the neighbor relationships. For instance, Najran is adjacent to Jazan and Asir, so these regions cannot share the same color.
- **Constraints:** The primary constraint is that adjacent regions must not share the same color. This ensures that no two neighboring regions are assigned the same color.

2.6.3 Arc-Consistency (AC-3)

The **AC-3 Algorithm** is used to enforce **arc-consistency** before solving the CSP. This process iteratively reduces the domains of variables by removing values that cannot satisfy the constraints. By doing so, inconsistent assignments are eliminated early, making the search process more efficient.

2.6.4 Backtracking Search

After enforcing arc-consistency, the problem is solved using **backtracking search**. This search method:

- Assigns a color to each region while ensuring the constraints are satisfied.
- Backtracks when a conflict arises (i.e., two adjacent regions share the same color) and tries a different assignment.
- Continues until a complete and valid solution is found, or all possibilities are exhausted.

2.6.5 Solution

The final solution is a valid coloring where:

- Each region is assigned a color from the domain {`Red, Green, Blue`}.
- No two adjacent regions share the same color.

2.6.6 Visualization

To represent the solution visually, the map is plotted using **Matplotlib**. Regions are displayed as color-coded points on a 2D map, with each point labeled according to its region. This visualization helps verify that the constraints are satisfied.

2.6.7 Conclusion

The solution approach effectively uses the **CSP framework** to model the Map Coloring problem and solves it using **arc-consistency** and **backtracking search**. The resulting solution satisfies all constraints and is visually verified through a plotted representation.

3 8-Queens Problem Using Genetic Algorithm

3.1 Overview

The 8-Queens problem is to place 8 queens on an 8x8 chessboard such that no two queens attack each other. Queens can attack along the same row, column, or diagonal.

Genetic Algorithms approach this problem by mimicking natural selection processes, evolving a population of potential solutions toward an optimal solution.

3.2 8-Queens Formulations

- Representation: Each individual is a list of 8 integers, where each integer represents the row position of a queen in each column.
- Fitness Function: The fitness of a solution is the number of non-attacking pairs of queens. A higher fitness indicates fewer conflicts.
- Selection: Use fitness-based selection to choose parents.
- Crossover: Implement a crossover function that combines two parents to produce offspring.
- Mutation: Randomly change the position of one queen in the board.

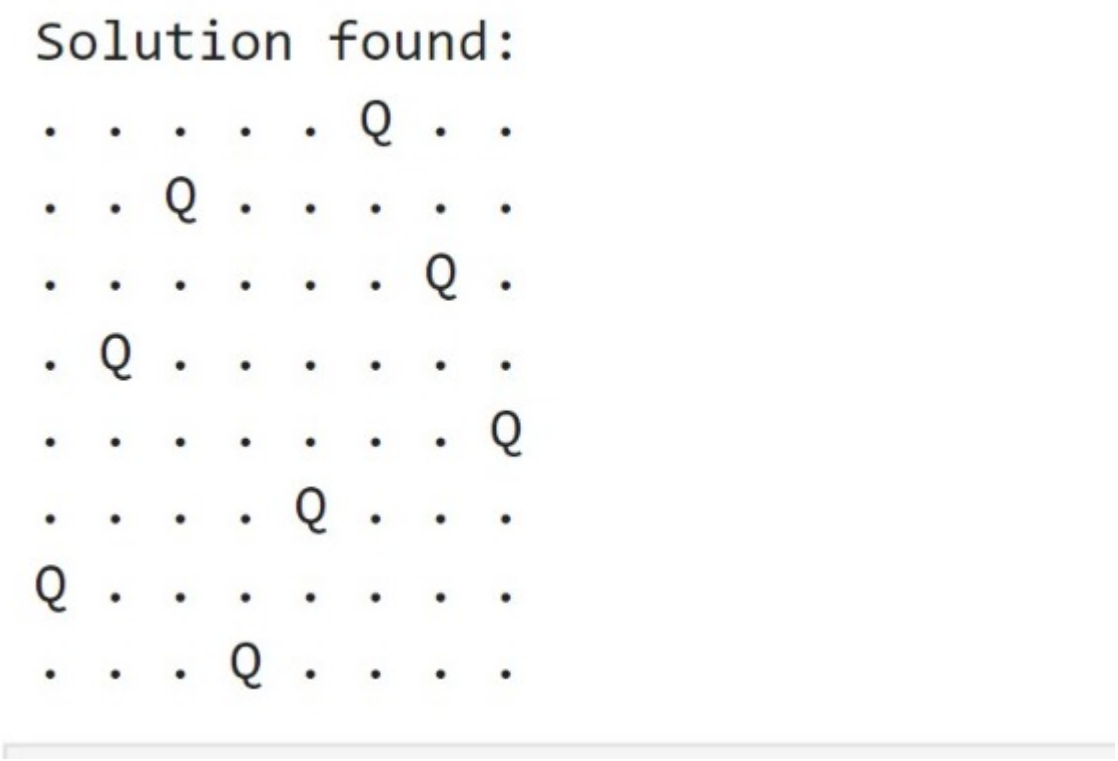
3.3 Code of the 8-Queens Problem: Genetic Algorithm Implementation

```
import random

# Fitness Function
def fitness(board):
    conflicts = 0
    for i in range(len(board)):
        for j in range(i+1, len(board)):
            if board[i] == board[j] or abs(board[i] - board[j]) == j - i:
                conflicts += 1
    return 28 - conflicts  # Max conflicts is 28, so higher fitness is better

# Selection: Fitness-based selection using roulette wheel
def select(population, fitness_values):
    total_fitness = sum(fitness_values)
    selection_probs = [f / total_fitness for f in fitness_values]
    selected_index = random.choices(range(len(population)), selection_probs)[0]
    return population[selected_index]

# Crossover: Single-point crossover
def crossover(parent1, parent2):
    point = random.randint(1, len(parent1) - 1)
    child = parent1[:point] + parent2[point:]
    return child

# Mutation: Random mutation of a single queen's position
def mutate(board):
    index = random.randint(0, len(board) - 1)
    new_position = random.randint(0, len(board) - 1)
    board[index] = new_position
    return board

# Main Genetic Algorithm
def genetic_algorithm(population_size, generations, mutation_rate):
```

```
    population = [random.sample(range(8), 8) for _ in range(population_size)]
    generation = 0

    while generation < generations:
        fitness_values = [fitness(individual) for individual in population]

        # Check for solution
        if max(fitness_values) == 28:  # A solution is found (zero conflicts)
            solution_index = fitness_values.index(max(fitness_values))
            solution = population[solution_index]
            return solution

        # Select parents
        new_population = []
        for _ in range(population_size):
            parent1 = select(population, fitness_values)
            parent2 = select(population, fitness_values)

            # Crossover
            child = crossover(parent1, parent2)

            # Mutation
            if random.random() < mutation_rate:
                child = mutate(child)

            new_population.append(child)

        population = new_population
        generation += 1

    # If no solution found within max generations, return the best solution
    best_solution = population[fitness_values.index(max(fitness_values))]
    return best_solution

# Display the board configuration
def display_board(board):
    for row in range(8):
        line = ""
        for col in range(8):
            if board[col] == row:
                line += "Q "
            else:
                line += ". "
        print(line)

# Run the Genetic Algorithm
population_size = 100
generations = 1000
mutation_rate = 0.05

solution = genetic_algorithm(population_size, generations, mutation_rate)
print("Solution found:")
display_board(solution)
```

Listing 3: Genetic Algorithm for 8-Queens Problem

3.4 Output



3.5 Explanation of the Solution Approach

Here is a detailed explanation of our GA approach:

3.5.1 Representation

Each individual (candidate solution) is represented as a list of 8 integers, where:

- Each index of the list represents a column on the chessboard.
- The value at each index represents the row position of the queen in that column.

For example, the representation $[0, 4, 7, 5, 2, 6, 1, 3]$ corresponds to queens being placed at:

- Column 0, Row 0
- Column 1, Row 4
- Column 2, Row 7
- and so on.

3.5.2 Fitness Function

The fitness function measures the quality of a solution. It calculates the number of non-attacking pairs of queens. The fitness is computed as:

- Start with a perfect score of 28 (maximum number of non-attacking pairs).

- Subtract points for conflicts:
  - Queens in the same row.
  - Queens on the same diagonal.

A fitness score of 28 indicates a valid solution with no conflicts.

3.5.3 Selection

Selection is performed using *fitness-based selection* (roulette wheel selection):

- Each individual is assigned a probability proportional to its fitness.
- Individuals with higher fitness have a higher chance of being selected as parents.

3.5.4 Crossover

Crossover combines the genetic information of two parent solutions to produce an offspring. A *single-point crossover* is used:

- Randomly choose a crossover point.
- Combine genes from the first parent up to the crossover point and from the second parent beyond that point.

For example:

- Parent 1: $[0, 4, 7, 5, 2, 6, 1, 3]$
- Parent 2: $[5, 3, 0, 7, 4, 1, 2, 6]$
- Crossover Point: 4
- Child: $[0, 4, 7, 5, 4, 1, 2, 6]$

3.5.5 Mutation

Mutation introduces randomness to maintain genetic diversity and avoid premature convergence:

- A random column (index) is selected.
- The queen in that column is moved to a new random row.

For example:

- Before Mutation: $[0, 4, 7, 5, 2, 6, 1, 3]$
- After Mutation: $[0, 4, 7, 2, 2, 6, 1, 3]$

3.5.6 Main Loop

The algorithm proceeds as follows:

1. **Initialization**: Start with a population of random solutions.
2. **Evaluation**: Compute the fitness for each individual.
3. **Termination Check**: If an individual has a fitness of 28, the solution is found.
4. **Reproduction**:
   - Select parents based on their fitness.
   - Perform crossover to create offspring.
   - Apply mutation with a specified probability.
5. Repeat until a solution is found or the maximum number of generations is reached.

Here, no two queens attack each other, demonstrating a valid configuration.

3.6 Advantages of Genetic Algorithm in 8-Queens

- **Efficiency**: GAs can converge quickly to solutions for complex problems.
- **Diversity**: Maintains a diverse population of solutions, avoiding local optima.
- **Adaptability**: Works well for various problem formulations.

4 Constraint Satisfaction Problem (CSP) vs Genetic Algorithm

Algorithm	**Constraint Satisfaction Problem (CSP)**	**Genetic Algorithm (GA)**
Approach	Deterministic search for a feasible solution by satisfying constraints.	Stochastic search using evolutionary techniques like selection, crossover, and mutation.
Problem Type	Well-defined problems with constraints (e.g., Map Coloring).	Optimization problems, often with no explicit constraints (e.g., 8-Queens).
Search Space Exploration	Systematic exploration using techniques like Backtracking Search or AC-3.	Randomized exploration by evolving a population of candidate solutions.
Solution Representation	Variables, domains, and constraints explicitly define the problem.	Solutions represented as chromosomes (e.g., lists, arrays).
Algorithm Behavior	Guarantees finding a solution if one exists.	May not guarantee an optimal solution; relies on convergence over iterations.
Efficiency	Computationally efficient for small or moderately sized problems.	Can handle large and complex search spaces but may be slower due to randomness.
Examples	Map Coloring Problem, Sudoku Solver.	8-Queens Problem, Traveling Salesperson Problem.

Table 1: Comparison between Constraint Satisfaction Problem (CSP) and Genetic Algorithm (GA).

5 Conclusion

In this assignment, we solved the Map Coloring Problem using Constraint Satisfaction Problem (CSP) algorithms, specifically Backtracking Search and Arc Consistency (AC-3), to find valid color assignments for each region of Saudi Arabia. Additionally, we solved the 8-Queens Problem using a Genetic Algorithm to find a solution where no two queens attack each other, leveraging techniques like selection, crossover, and mutation. This process allowed us to explore the application of both CSP and Genetic Algorithms in solving complex optimization and constraint-based problems. And helped us better understand the application of search and optimization algorithms in solving complex problems.

6 Knowledge Acquired

Completing this assignment has deepened our understanding of the practical applications of search algorithms in solving real-world problems. The Map Coloring Problem demonstrated CSPs' power in systematically addressing constraints, while the 8-Queens Problem showcased the adaptability of Genetic Algorithms in evolving solutions through selection, crossover, and mutation. Both implementations highlighted the balance between computational efficiency and algorithmic creativity.

What We Learned:

- Algorithm Characteristics: Each algorithm has unique properties that make it suitable for different problems.
- Efficient Search: We understood the importance of reducing search space using constraints in CSP, which will lead to faster and more effective problem-solving

- Programming and Problem Solving: We learned to structure code for clarity and maintainability. And to debug and test solutions to validate their correctness.

This assignment has improved our problem-solving skills and helped us understand search algorithms better, giving us useful knowledge for future artificial intelligence projects.